

chapter14

제14장 통합 사례 연구: 공공 실시간 민원/재난 시스템

지금까지 이 교재는 부품을 하나씩 만들었다 — 수집과 품질(1~2장), 아키텍처(3장), 메시지 신뢰성(4~5장), 스트림과 배치(6~7장), 피처(8장), 모델의 등록·배포·감시(9~11장), 그리고 부하와 거버넌스(12~13장). 각 장의 실습은 의도적으로 작았고, 서로 떨어져 있었다. 이 장은 그 부품들을 하나의 운영 흐름으로 재조립한다: 민원 이벤트가 실시간으로 흘러들고, 하루가 마감되면 집계가 확정되고, 확정된 건수가 다음 날 예측의 입력이 되고, 다음 날의 실제 건수가 어제의 예측을 채점하는 — 작지만 완결된 공공 민원 예측 시스템이다. 새 기술은 하나도 등장하지 않는다. 대신 두 가지가 새로 등장한다. 첫째는 **연결부**: 부품과 부품이 만나는 곳에서만 생기는 고장(중복 재전송, 오염 이벤트, 의존 서비스의 중단)을 실측으로 드러내고 대응을 리허설한다. 둘째는 **인수의 관점**이다: “이 시스템을 발주자가 검수할 수 있는가”라는 질문에, 사람의 주장이 아니라 실행 증거에서 자동 생성되는 인수 검수표로 답한다. 실습(★★★★☆)은 Docker Compose로 전체 스택을 띄워 3일치 운영을 시뮬레이션하고, 장애 드릴(drill) 3종을 수행하며, 16개 항목의 인수 검수표를 전향 통과시킨다.

학습 목표

DE와 MLOps의 부품을 엔드투엔드(end-to-end) 운영 흐름으로 설계·연결한다.
연결부의 고장(중복·오염·부분 실패)을 격리하는 원칙을 실측으로 설명한다.
장애 시나리오와 대응 절차를 runbook으로 문서화하고 드릴로 검증한다.
예측 시스템이 정책 의사결정 지원·정책 분석·평가와 연결되는 경계를 설계한다.
앞 장들이 각각 정한 계층별 결정을 한 표로 모아 서로 충돌하는 지점을 찾고, 그 조합이 깨지는 조건을 적을 수 있다.

14.1 시스템 요구사항 정의

학습 목표

기능 요구와 비기능 요구를 분리하고, 각각의 검증 방법을 명세한다.
“실시간”의 의미를 계약 가능한 언어로 좁힌다.

가상 발주 상황

이 장의 시스템은 다음 발주 요구에서 출발한다 — “구 단위 민원 접수를 실시간으로 수집·집계하고, 다음 날 민원 규모를 예측해 자원 배치 판단을 지원하는 시스템.” 문장 하나지만, 계약과 검수의 언어로 옮기면 요구는 두 층으로 나뉜다.

기능 요구는 시스템이 “무엇을 하는가”다: 민원 이벤트 수집, 정제와 표준코드 매핑, 일별 확정 집계, 익일 예측, 예측의 사후 채점. **비기능 요구**는 그 일을 “어떤 조건에서도 하는가”다: 이벤트를 유실하지 않는다, 중복 접수·재전송이 집계를 오염시키지 않는다, 구성 요소 하나의 장애가 전체를 멈추지 않는다, 모든 판단이 사후 감사 가능하다, 폐쇄망에서도 배포된다. 실무 발주 문서에서 사고가 발생하는 쪽은 대부분 후자다 — 기능은 시연으로 확인되지만, 비기능은 **고장을 의도적으로 유발하기 전에는** 확인되지 않기 때문이다(14.5의 드릴이 그 확인 절차다).

표 14.1 요구사항 정의표(요구 → 검증 방법 → 근거 장)

구분	요구	검증 방법(이 장의 실측)	근거 장
기능	이벤트 수집(스트림)	3일치 60건 발행→소비, 일별 물리 수신 20/28/22	4장
기능	정제·표준코드 매핑	매핑 18/16/20, 미기재· 미매핑 분리 보고	2·7장
기능	일별 확정 집계	_SUCCESS 마커 3개, 7장 확정치와 일치	7장
기능	익일 예측	champion 호출 9건 전부 응답(전부 6.0)	9·10장
기능	예측 사후 채점	지연 MAE 일별 0.6667/1.3333 기록	11장
비기능	중복 무해(역등)	재전송 10건 주입 → 집계 불변	5장
비기능	오염 격리	파싱 불가 1건 → 격리 큐, 파이프라인 지속	2장
비기능	부분 실패 격리	모델 API 중단 중에도 집계 확정, 예측만 대기	이 장
비기능	복구 수렴	재기동+재시도 → 대기 0건	이 장
비기능	감사 가능성	모델 신원·데이터 지문· 판단 기록이 값으로 남음	9장
비기능	폐쇄망 배포	모델을 이미지에 고정, 실행 중 외부 참조 없음	10장

이 표의 셋째 열이 이 장의 전부다. 요구마다 “어떻게 확인하는가”가 붙어 있고, 그 확인이 전부 실습 14.1에서 실제로 수행된다 — 요구사항 정의와 인수 검수표(표 14.10)는 같은 표의 앞면과 뒷면이다.

“실시간”의 정확한 정의

발주 문서의 “실시간”은 그대로 두면 분쟁의 원인이 된다. 이 시스템의 실시간은 두 속도로 나뉜다: **접수는 스트림**(이벤트가 오는 대로 수신·반영 — 초 단위)이고, **확정과 예측은 일 배치**(하루가 마감되어야 집계와 확정이 나간다)다. 3장에서 배운 하이브리드 구조가 정확히 이것이다 — 같은 시스템 안에 두 개의 시계가 있고, 어느 산출물이 어느 시계를 따르는지 계약서에 적혀야 한다. 재난 대응 관점이라면 요구가 달라진다: 침수 신고의 집계는 “일 마감”을 기다릴 수 없으므로 분 단위 창(6장의 윈도우 집계)이 필요하고, 그 순간 아키텍처 선택지가 달라진다. 같은 부품, 다른 시계 — 요구사항 정의가 아키텍처보다 먼저인 이유다.

시계를 나누는 이 판단은 공공만의 것이 아니다. 이커머스의 주문 파이프라인도 매출·환불·반품을 일 단위로 마감해 확정하고, 결제 사기 탐지는 초 단위로 판정하며, 물류 재고는 시간 단위로 갱신한다. 바뀌는 것은 주기를 정하는 근거다. 공공에서는 행정 주기와 감사 단위가 주기를 정하고, 민간에서는 손실이 얼마나 빨리 커지는가가 정한다 — 사기 거래는 초 단위로 손실이 쌓이므로 초 단위 판정이 필요하고, 재고는 시간 단위로 움직이므로 그보다 빠를 이유가 없다. 계약의 형태는 어느 쪽이든 같다. 민간에서는 이벤트 발생부터 산출물 확정까지의 시간을 E2E 지연으로 정의해 SLA(Service Level Agreement, 서비스 수준 계약)에 적는다. 표 14.1의 셋째 열이 그 SLA를 무엇으로 확인할지를 적어 둔 자리다.

공공 사례

공공 정보시스템의 구축·운영은 산출물과 검사 절차를 규정한 행정규칙(행정안전부, 행정기관 및 공공기관 정보시스템 구축·운영 지침)의 적용을 받는다. 그러나 이런 일반 규범에는 AI 특화 항목 — 모델 신원 확인, 드리프트 감시, 재학습 조건 — 이 아직 명세되어 있지 않다. 표 14.1처럼 AI 요소의 비기능 요구와 검증 방법을 발주 문서에 명시적으로 추가하는 일은, 현재로서는 발주자와 수행사가 스스로 채워야 하는 공백이다. ### 핵심 정리 - 요구는 기능(무엇을)과 비기능(어떤 조건에서도)으로 분리하고, 각각에 검증 방법을 붙인다 — 검증 방법이 없는 요구는 검수할 수 없다. - 비기능 요구는 고장을 의도적으로 유발해야 확인된다 — 드릴(14.5)이 검증 절차다. - “실시간”은 시계를 명시해 계약 가능한 언어로 좁힌다: 접수는 스트림, 확정·예측은 일 배치.

14.2 데이터 수집, 처리, 피처, 모델, 배포, 모니터링 통합

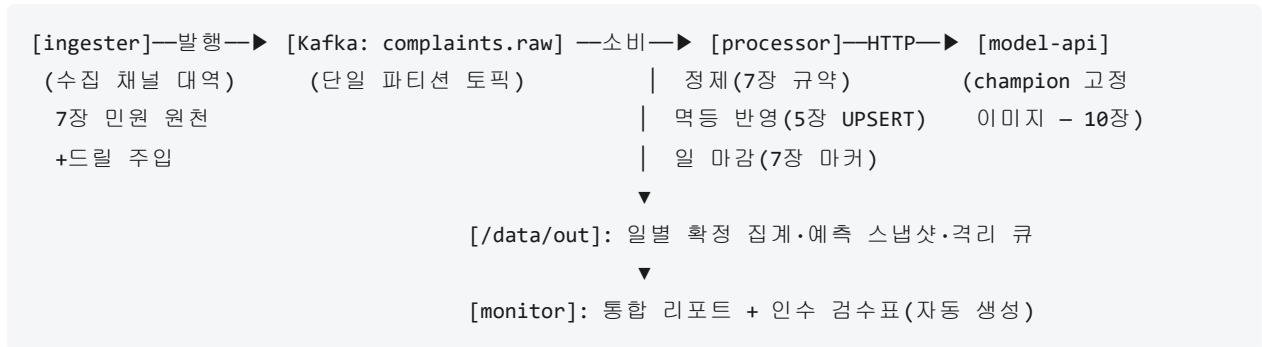
학습 목표

앞 장의 부품이 통합 MVP의 어느 자리에 들어가는지 매핑한다.

“재조립이 원본과 같은 값을 낸다”를 다섯 개의 교차 검증으로 확인한다.

아키텍처: 부품의 재조립

통합 MVP의 구조는 다음과 같다. 모든 연결은 Docker Compose의 내부 네트워크 안에 있고, 호스트에 노출되는 것은 모델 API 포트 하나뿐이다.



ML 시스템에서 모델 코드는 전체의 작은 부분이고 시스템의 유지 부담은 주변 구성 — 데이터 흐름, 연결, 설정 — 에서 온다는 것은 이 분야의 고전적 관찰이다(Sculley et al., 2015). 이 그림이 그 관찰의 실물이다: 모델은 오른쪽 끝의 상자 하나이고, 나머지 전부가 모델에게 올바른 입력을 만들어 주고 출력을 안전하게 소비하는 장치다.

표 14.2 컴포넌트 통합 매핑(부품 → 원 장 → MVP에서의 형태)

부품	원 장	통합 MVP에서의 형태	축소된 것
이벤트 수집	1-4장	ingester가 7장 민원 원천을 Kafka로 발행	실제 접수 채널 → 파일 재생
메시지 버스	3-4장	Kafka+Zookeeper(3장 이미지 재사용), 단일 파티션	3파티션 → 1(마감 순서 보장)
중복 방지	5장	processor의 complaint_id UPSERT(SQLite)	동일 — 5장 규약 그대로
정제·매핑	2-7장	공백·약칭 보정, 표준코드 매핑, 4갈래 분리 보고	동일 — 7장 규약 그대로
일 마감·확정	7장	day_close 제어 이벤트 → 집계 확정 → <code>_SUCCESS</code>	Airflow 스케줄 → 제어 이벤트
피쳐	8장	당일 확정 건수 = 익일 예측의 입력(전일 건수)	Feast → 집계 산출물 직접 계산
모델·레지스트리	9장	bootstrap이 동일 지문 데이터로 재구성, champion 승격	공용 추적 서버 → 로컬 SQLite
서빙	10장	app.py 바이트 동일 복사 + champion 고정 이미지	shadow는 off(단순화)
감시	11장	지연 레이블 채점(일별 MAE)·운영 카운터·검수표	대시보드 → JSON 리포트

흐름을 한 바퀴: 하루의 일생

1일차(2026-07-01)의 처리 순서는 다음과 같다. ingester가 민원 20건을 토픽에 발행한다. processor는 이벤트가 오는 대로 정제한다 — 앞뒤 공백 1건 보정, 약칭("강남"→"강남구") 4건 보정, 지역 미기재 1건과 표준코드에 없는 지역("판교") 1건은 폐기하지 않고 분리 기록, 그리고 complaint_id를 키로 UPSERT(재수신이면 업무 상태는 그대로 두고 관찰 횟수만 올린다). 여기까지가 스트림 처리 경로에서 수행하는 작업이다.

이어서 ingester가 일 마감 제어 이벤트(`day_close`)를 발행한다. processor는 마감을 받으면 그날의 확정 집계를 쓴다 — 마포 5·관악 5·강남 8, 매핑 총 18건. 품질 회계(물리 20 = 고유 20 + 중복 0, 고유 20 = 매핑 18 + 미기재 1 + 미매핑 1)를 기록하고, 마지막에 `_SUCCESS` 마커를 쓴다(7장 규약:

마커는 항상 마지막). 그리고 확정된 오늘 건수를 입력으로 모델 API를 호출해 **내일**의 예측을 만든다 — 오늘의 확정 건수가 곧 내일 예측의 “전일 건수” 피쳐다(8장의 의미론: 예측 시점에 알 수 있는 값만 쓴다). 여기까지가 배치의 시계다.

다음 날(7/2) 마감에서 두 가지가 일어난다. 7/2의 집계가 확정되면서, **어제 만든 7/2 예측 3건이 채점된다** — 실제 건수(레이블)가 도착한 것이다. 11장에서 “레이블은 하루 늦게 도착한다”고 했던 지연 레이블 평가가 이 시스템에서는 정확히 하루 지연으로 실체화된다. 그리고 7/2 확정 건수로 7/3의 예측이 나간다. 이 주기 — 확정, 채점, 예측 — 가 매일 반복되는 것이 시스템의 기본 운영 흐름이다.

이벤트 한 건의 여정: C260702-016

통합을 이해하는 가장 효과적인 방법은 이벤트 하나를 끝까지 추적하는 것이다. 7/2의 실제 이벤트 한 건 — `{"complaint_id": "C260702-016", "region_raw": "관악", "category": "도로", ...}` — 의 여정을 상태 DB와 증거 파일의 실측으로 재구성하면 다음과 같다.

역(station)	무슨 일이 일어났나(실측)	근거 장
① 발행	ingester가 토픽에 발행. 이 건은 “마지막 10건”에 포함되어 한 번 더 발행됨(드릴 D2)	4장
② 첫 수신·정제	공백 보정 없음(ws_stripped 0), 약칭 보정 “관악” → “관악구”(alias_fixed 1), 표준코드 11620 매핑	2·7장
③ 먹등 반영	complaint_id 키로 INSERT, seen_count=1	5장
④ 재수신	같은 키 재도착 → UPSERT: 업무 상태 불변, seen_count=2 (상태 DB 실측)	5장
⑤ 일 마감	7/2 확정 집계에서 관악구 5건 중 1건으로 계수	7장
⑥ 피쳐화	관악구 확정 5건이 그대로 7/3 예측의 입력(전일 건수 5)	8장
⑦ 예측	champion 호출: 입력 5 → 6.0(champion v1·local-dir 신원과 함께 기록)	9·10장
⑧ 채점	7/3 마감에서 관악구 실제 5건 도착 → 절대 오차 $ 6.0 - 5 = 1.0$ 기록	11장

여덟 개 역 어디에도 14장이 새로 만든 규칙이 없다 — 전부 앞 장의 규약이고, 14장이 한 일은 역들을 이어 붙인 것뿐이다. 같은 7/2 이벤트 중 대조 사례 둘도 실측으로 남아 있다: C260702-001은 재전송이 아니라 **원천 파일 안에서 두 번 접수**된 건인데 소비자 쪽 기록은 똑같이 seen_count=2다(수송 중복과 업무 중복이 같은 장치로 흡수됨 — 14.5 드릴 D2). C260702-017(“성남시”)은 두 번 수신되었지만 두 번 다 표준코드 매핑에 실패해 미매핑 1건으로 분리 보고되었다 — 중복 흡수와 매핑 실패가 독립적으로, 각자의 자리에 기록된다.

재조립의 증명: 다섯 개의 교차 검증

통합에서 가장 흔한 의문은 “옳거 붙이는 과정에서 무엇인가 달라지지 않았는가”이다. 이 실습은 그 질문에 값으로 답한다 — 앞 장들이 커밋해 둔 실측과 이 장의 실행이 일치하는지를 오케스트레이터와 monitor가 자동 대조한다.

표 14.3 재조립 교차 검증 5종(전부 실측 일치)

#	검증 항목	기대값(원 장의 커밋 실측)	이 장의 실측	판정
1	서빙 앱 동일성	10장 app.py sha256 ec7ec653...	복사본 sha256 일치	일치
2	훈련 데이터 동일성	9장 피쳐 지문 96f6b6b3...	bootstrap 지문 일치(불일치 시 즉시 실패)	일치
3	집계 재구현	7장 확정치 3일 (마포5·관악5·강남8 등)	스트림 경로 집계 동일	3일 모두 일치
4	예측 동일성	9·10장 champion 예측 6.0	호출 9건 모두 6.0	일치
5	평가 동일성	9장 훈련 MAE 1.0	지연 채점 6건 전체 MAE 1.0	일치

검증 3이 특히 주목할 부분이다. 7장은 같은 원천을 **배치**(Airflow 태스크의 파일 처리)로 집계했고, 이 장은 **스트림**(Kafka 소비 후 마감 집계)으로 집계했다 — 경로가 완전히 다른데 확정치가 같다. 처리 방식이 바뀌어도 업무 규칙(정제 4갈래·먹등 키·집계 기준)이 같으면 결과가 같아야 한다는 것, 그리고 그것을 수치 대조로 **증명할 수 있다**는 것이 이 검증의 요지다. 람다 아키텍처(3장)의 대표적 문제 — 배치 경로와 스트림 경로의 결과 불일치 — 를 검증 가능한 형태로 확인한 것이기도 하다.

이 다섯 검증을 소프트웨어 공학의 어휘로 옮기면 **계약 테스트(contract test)**의 사고방식에 가깝다 — 더 정확히는 과거에 기록해 둔 정답값과 현재 산출물을 대조하는 **골든 마스터(golden master)·회귀 검증**이다. 계약 테스트란 두 구성 요소가 만나는 경계에서 “생산자가 내놓는 것과 소비자가 기대하는 것이 일치하는가”를 각자 독립적으로 검증하는 기법으로(Fowler, *ContractTest*; 소비자 주도 계약의 구현으로 Pact가 널리 쓰인다), 원래는 서비스 경계의 메시지·응답 약속을 다룬다. 표 14.3의

각 행은 그 사고방식을 시간 축으로 옮긴 것이다 — 서빙 앱(생산자)의 sha256이 10장이 커밋한 값(소비자의 기대)과 같아야 하고, 집계기(생산자)의 확정치가 7장이 커밋한 값과 같아야 한다. 다른 점은 계약의 상대가 다른 팀이나 다른 서비스가 아니라 **과거의 자기 자신**(앞 장이 커밋해 둔 실측)이라는 것이다. 이 장은 그 약속을 통합의 접합면이 아니라 **시간의 접합면**(9·10장 → 14장)에서 지켜, “옮겨 붙이는 동안 값이 변하지 않았다”를 자동으로 확인한다. 적용 자격을 분명히 하면 — 이는 계약 테스트라는 방법론의 사고방식을 차용한 것이지 Pact 같은 특정 도구의 직접 선례는 아니며, 소비자의 기대값과 생산자의 산출물을 별도 계약 파일로 명시해 관리하고 재조립 검증을 CI 게이트로 상설화하면(표의 위험 식별 참조) 정식 계약 테스트에 더 가까워진다.

같은 대조를 민간에서는 데이터 파이프라인을 갈아탈 때 쓴다. 웨어하우스를 옮기거나 배치를 스트림으로 바꿀 때, 원본과 대상의 집계를 맞춰 보고 불일치가 나오면 배포를 자동으로 막는다(회귀 검증). 절차는 표 14.3과 같고, 결과를 어떻게 다루는지만 다르다 — 공공은 값이 일치했다는 기록을 감사 증빙으로 보관하고, 민간은 배포 차단 of 근거로 사용한 뒤 폐기한다.

검증 5는 계산상 필연이다. 채점된 예측 6건(7/2·7/3 각 3건)은 9장의 훈련 쌍 6개와 같은 데이터이고 모델도 같으므로, 지연 평가의 전체 MAE는 훈련 MAE와 같을 수밖에 없다 — $(1+1+0+0+1+3)/6 = 1.0$. 운영 감시 경로가 훈련 평가와 같은 값을 재현하는지 확인하는 것은, 감시 코드 자체의 정합성 검증(11장 실습이 2장 값을 재현했던 것과 같은 원리)이다.

피처 스토어 없이 피처 일관성은 어디서 오는가

이 MVP에는 8장의 Feast가 없는데, 그렇다면 훈련-서빙 일관성(8장의 존재 이유)은 무엇이 보장하는가? 답은 **경로의 단일화**다: 훈련 데이터(9장 쌍)도, 서빙 입력도, 같은 “일별 확정 집계”에서 나온다. 확정 집계가 피처의 단일 진실 원천이고, `_SUCCESS` 마커가 “이 값은 이제 피처로 써도 된다”는 신호다. 피처가 한 종류(전일 건수)이고 소비자가 하나(이 모델)라서 가능한 축소이며 — 피처가 수십 개, 소비 모델이 여럿이 되는 순간 이 암묵적 규약은 성립하지 않게 되고 8장(피처 스토어)과 13장(피처 거버넌스)의 장치가 필요해진다. 축소가 성립하는 조건을 아는 것이 축소를 적용할 자격이다.

계층별 결정은 서로 모순되지 않아야 한다

통합에서 실제로 발생하는 문제는 부품의 결함이 아니라 계층별 결정의 불일치다. 앞 장들은 각자의 맥락에서 값을 하나씩 정했다 — 4장은 파티션 수와 파티션 키를, 5장은 중복 제거 키를, 6장은 창과 워터마크를, 7장은 배치 주기와 마커 규약을, 9장은 승격 게이트를, 11장은 감시 임계와 판정 주기를, 13장은 접근 등급을 정했다. 각 값은 자기 장 안에서 근거가 분명했다. 그 값들을 한 시스템에 모으면 서로를 무효로 만드는 조합이 나온다.

이 MVP에서 실제로 걸린 조합이 셋이다. 첫째, 4장은 처리량을 위해 파티션 3개를 권했지만, 7장의 일 마감은 그날 이벤트가 전부 도착한 뒤에 마감 신호가 온다는 것을 전제한다. 파티션이 여럿이면 그 전제가 깨진다. 그래서 이 MVP는 파티션을 1개로 줄였다 — 처리량을 마감 순서와 맞바꾼 것이고, 표 14.5가 그 맞바꿈을 기록한다. 둘째, 9장의 승격 게이트는 MAE 1.05인데 11장의 지연 채점은 하루 표본이 3건이다. 표본 3건짜리 MAE로 게이트를 판정하면 하루의 우연이 재학습을 유발한다 — 7/3의 1.3333이 정확히 그 경우다. 게이트 값을 바꾸거나 판정 주기를 늘리거나 둘 중 하나는 조정해야 하며, 이 MVP는 후자를 택해 그 값을 감시 강화 신호로만 읽는다. 셋째, 5장의 멍등

키는 `complaint_id` 이고 이 장의 예측 재시도 키는 (대상일, 지역)이다. 서로 다른 층의 결정인데 둘 다 먹등이라서 D3의 재시도가 안전했다. 한쪽이라도 먹등이 아니었으면 복구 절차 자체가 새 오염을 만들었을 것이다.

세 조합의 공통 형태는 하나다 — 한 계층의 최적값이 다른 계층의 전제를 깬다. 그래서 통합 설계의 첫 작업은 새 값을 정하는 것이 아니라, 이미 정해진 값들을 한 표에 모아 충돌을 찾는 것이다(실습 14.1의 1단계).

공공 사례

공공 시스템 감리에서 “구성 요소별 시험은 통과했으나 통합 시험에서 실패”는 흔한 지적 사항이다. 표 14.3의 접근 — 부품별 기준값을 먼저 확정하고, 통합 후 같은 값이 재현되는지 자동 대조 — 은 통합 시험을 “시나리오가 돌아다”에서 “값이 일치했다”로 끌어올린다. 검수 관점에서 둘의 증거 능력은 다르다: 전자는 화면, 후자는 기록이다. ### 핵심 정리 - 통합은 재발명이 아니라 재조립이다 — 서빙 앱은 10장 파일의 바이트 동일 복사본이고, 코드가 그 동일성을 확인한다. - 재조립의 품질은 교차 검증으로 증명한다: 데이터 지문·집계 확정치·예측값·평가값이 원 장의 커밋 실측과 일치 (표 14.3). - 확정 집계는 피처의 단일 진실 원천이다 — 이 축소는 피처 1종·소비자 1개일 때만 성립하며, 그 너머는 8·13장의 장치가 맡는다. - 통합의 첫 작업은 새 값을 정하는 것이 아니라 앞 장들이 정한 값을 한 표에 모아 충돌을 찾는 것이다 — 파티션 수 대 마감 순서, 게이트 값 대 표본 크기가 그 예다.

14.3 정책 의사결정 지원과 정책 분석·평가 시스템의 연결

학습 목표

예측 산출물이 사람의 의사결정으로 넘어가는 경계를 설계한다.
운영 로그가 정책 분석·평가의 원자료가 되는 구조를 이해한다.

예측의 소비자는 시스템이 아니라 사람이다

이 시스템의 마지막 산출물 — “내일 강남구 민원 약 6건” — 은 어디로 가는가. 기술적으로는 JSON 한 줄이지만, 제도적으로는 여기가 가장 중요한 경계다. 이 교재의 일관된 원칙(1장부터)은 공공 AI가 **정책 결정 시스템**이 아니라 **정책 의사결정 지원**이라는 것이다: 예측은 자원 배치·순찰 강화·안내 발송 같은 판단의 **입력**이 되고, 판단과 집행은 권한과 책임이 있는 사람이 한다.

경계 설계의 세 요소를 이 MVP에 대입하면 다음과 같다.

자동 집행 금지 지점: 예측이 임계를 넘어도 시스템은 아무것도 집행하지 않는다 — 예측 스냅샷에 기록될 뿐이다. 자동으로 해도 되는 것(집계 확정, 예측 기록, 재시도)과 사람 승인이 필요한 것(자원 이동, 대민 안내)의 목록이 운영 계획서에 명시되어야 한다.

사람 개입 지점: 예측을 보는 사람이 “왜 6건인가”를 물을 때 시스템이 내놓을 수 있는 근거. 한계는 다음과 같다 — champion(v1)은 평균 예측기라서 근거가 “최근 확정 집계계의 평균”이 전부다. 입력(전일 8건)이 무엇이든 6.0을 답한다. 빈약하지만 **설명 가능한** 빈약함이며, 복잡한 모델로 바꾸는 순간 정확도와 설명가능성의 긴장(16장)이 시작된다.

평가지표 주입: 어떤 지표로 이 시스템을 계속 신뢰할지(지연 MAE·게이트 값)는 운영 조직이 아니라 평가 기준을 관장하는 쪽이 정한다. 재학습 게이트(9장 1.05)와 감시 임계(11장)가 그 지표의 실체이고, 17장의 Policy Methodology Lab이 이 기준을 관리하는 조직 형태다.

운영 로그가 정책 평가의 원자료가 된다

이 시스템이 매일 남기는 기록 — 예측(6.0), 실제(5·5·6...), 절대 오차, 일별 MAE — 을 다시 확인한다. 이것은 운영 감시 자료인 동시에, “이 예측 서비스가 행정에 도움이 되었는가”를 묻는 **정책 평가의 원자료**다. 실측에서 7/3의 일 MAE는 1.3333으로 9장 승격 게이트(1.05)를 넘었다(증거 JSON의 `gate_exceeded_days`). 운영의 언어로는 “감시 강화 신호”(단, 하루치 표본 3건 — 11장의 소표본 주의가 그대로 적용된다)이고, 평가의 언어로는 “이 도구의 유효성을 어느 주기·어느 표본으로 재판정할 것인가”라는 질문이다. 같은 숫자가 두 시스템 — 운영과 평가 — 에 다르게 소비되며, 그 연결 통로(누가, 어느 주기로, 어떤 형식의 기록을 받는가)가 설계 대상이다.

주의할 것: 예측 오차의 평가는 **정책효과 평가가 아니다**. “민원 건수를 잘 맞혔다”와 “이 예측으로 행정이 개선되었다”는 다른 질문이고, 후자는 개입·비교집단·시차를 다루는 정책 분석의 방법론을 요구한다. MLOps의 지표를 정책효과의 증거로 승격시키는 비약이 공공 AI 보고서의 흔한 오류다.

표 14.4 정책 AI 통합 아키텍처 검토표(정본 산출물)

검토 항목	확인 질문	이 MVP의 답	근거
산출물의 지위	예측은 집행되는가, 참고되는가	기록만 — 집행 경로 없음	14.3 경계 설계
자동/승인 구분	자동 허용 작업 목록이 있는가	집계·기록·재시도만 자동	runbook(표 14.7)
근거 제시	“왜 이 값인가”에 답할 수 있는가	평균 예측기 — 근거 단순·명시	9장 모델 카드 항목
신원·감사	지금 어느 모델인가를 물을 수 있는가	/health가 이름·버전·출처 응답	10장 실측
평가지표 주입	게이트·임계를 누가 정하는가	9장 게이트 1.05(승격 기준 문서화)	9·11장, 17장 예고
평가 원자료	예측·실제·오차가 기록되는가	예측 스냅샷 + 일별 MAE	이 장 실측
오차평가≠효과평가	두 평가가 구분되어 있는가	본문 명시 — 효과평가는 별도 방법론	14.3
개인정보 경계	민원 원문이 예측 경로에 들어가는가	집계(건수)만 — 원문 미사용	2·3장

공공 사례

민원 예측을 도입한 기관이 “예측 정확도 보고서”만 남기는 경우와, “예측→판단→조치→결과”의 연쇄 기록을 남기는 경우를 대비하면 차이가 나타난다. 전자는 모델 평가이고 후자가 정책 평가의 시작이다. 이 장의 예측 스냅샷은 전자를 자동화한 것이며, 후자로 가려면 “그 예측을 보고 무엇을 결정했는가”라는 사람 쪽 기록이 연결되어야 한다 — 15장의 질의 로그와 human-in-the-loop 승인 흐름이 그 연결의 LLM판이다. ### 핵심 정리 - 예측은 판단의 입력이다 — 자동 집행 금지 지점과 사람 개입 지점을 목록으로 명시한다(정책 결정이 아니라 정책 의사결정 지원). - 운영 로그(예측·실제·오차)는 정책 평가의 원자료이며, 평가 기준(게이트·임계)은 운영 밖에서 주입된다(17장 Policy Methodology Lab). - 예측 오차 평가와 정책효과 평가를 혼동하지 않는다 — 후자는 별도의 방법론이다.

14.4 MVP 범위와 제외 범위

학습 목표

MVP의 포함/제외를 근거와 함께 명시하는 범위 표를 작성한다.
제외 항목마다 “운영 승격 시 무엇으로 대체되는가”를 답한다.

범위 표가 없는 MVP는 “덜 만든 것”이다

MVP(Minimum Viable Product)의 가치는 작음이 아니라 **경계의 명시**에 있다. 무엇을 뺐는지, 왜 뺐는지, 언제 다시 넣는지가 문서로 남으면 “작게 만든 것”이고, 없으면 그냥 “덜 만든 것”이다. 발주. 검수의 언어로 말하면 범위 표는 인수 분쟁을 사전에 차단하는 장치다 — 검수일에 “왜 대시보드가 없느냐”는 질문은, 범위 표에 “감시 시각화: 제외, JSON 리포트로 대체, 운영 승격 시 3장 Grafana”라고 적혀 있을 때에만 1분짜리 대화가 된다.

표 14.5 통합 MVP 포함/제외 범위표

항목	판정	이 MVP에서의 대체물	원 장	운영 승격 시
이벤트 버스 (Kafka)	포함	단일 브로커·단일 파티션	3·4장	다중 파티션·복제(키 설계는 4장)
먹등 소비	포함	complaint_id UPSERT	5장	동일 원칙, 저장소만 교체
일 마감·확정	포함	day_close 제어 이벤트	7장	스케줄러(Airflow)가 마감 발행
모델 서빙	포함	champion 고정 이미지 1대	10장	복수 인스턴스 + 무중단 교체
자연 채점	포함	마감 시 자동 채점·MAE	11장	감시 파이프라인·알림 연동
워크플로 스케줄러	제외	오케스트레이션 스크립트	7장	Airflow(실패 전파·재실행 이력)
피처 스토어	제외	확정 집계 직접 계산	8장	피처 다중·다소비자 시 Feast
공용 추적 서버	제외	로컬 SQLite bootstrap	9장	팀 공용 MLflow 서버
감시 시각화·알림	제외	JSON 리포트·콘솔	3·11장	Prometheus·Grafana-on-call
스트림 윈도우 집계	제외	일 마감 배치 집계	6장	분 단위 요구(재난) 시 워터마크
부하 대응·스케일링	제외	실측 없음(정직 표기)	12장	부하 테스트로 한계 실측 후 설계
피처 거버넌스	제외	피처 1종 — 규약 암묵	13장	카탈로그·접근 로그·변경 관리
shadow 평가	제외	challenger off	10장	후보 모델 병행 평가 재개
실제 알림 전송	제외	판정 기록까지만	11장	메신저·문자 연동 + 알림 피로 설계

제외 열의 공통점은 다음과 같다 — 전부 “규모가 커질 때 필요한 것”이다. 파티션 하나, 피처 하나, 모델 하나, 소비자 하나인 동안 이 축소들은 안전하다. 각 행의 “운영 승격 시” 열이 그 축소가 깨지는 조건의 목록이며, 이 표를 유지·갱신하는 것 자체가 운영 문서의 일부다.

한 가지 표기: 재난 상황의 트래픽 급증 — 평시의 수십 배 이벤트가 몰리는 상황 — 에서 이 MVP가 버티는지는 **이 장에서 실측하지 않았다**. 그 실측(부하를 걸어 한계와 병목을 측정하는 일)은 12장의 방법론이고, 급증 대응 없이 재난 시스템을 표방할 수 없다는 점을 범위 표가 명시한다(보충 참조).

공공 사례

공공 사업의 단계별 구축(1차 MVP → 2차 고도화)에서 가장 자주 깨지는 것이 범위의 기억이다. 1차 검수 시점의 구두 합의("그건 2차에서")는 담당자가 바뀌면 사라진다. 표 14.5 형식 — 항목·판정·대체물·승격 조건 — 을 사업 산출물로 남기면, 2차 발주의 요구사항 정의서가 절반은 이미 쓰여 있는 셈이다. ### 핵심 정리 - MVP의 가치는 경계의 명시다 — 제외 항목마다 근거·대체물·복귀 조건을 적는다(표 14.5). - 축소가 성립하는 조건(단일 파티션·피쳐 1종·모델 1개)을 알고 있어야 축소를 적용할 자격이 있다. - 부하 급증 대응은 이 장에서 실측하지 않았음을 명시적으로 표기한다 — 사전 부하 실측은 12장의 방법론이다.

14.5 장애 시나리오와 대응 절차

학습 목표

연결부 고장 3종을 드릴로 재현하고 격리·복구를 실측으로 확인한다.
runbook을 "장애 중에 읽는 문서"가 아니라 "평시에 검증하는 문서"로 설계한다.

직관: 통합의 고장은 연결부에서 난다

부품별 실습(4~11장)에서 각 부품은 이미 검증되었다. 통합 후에 새로 생기는 고장은 부품 안이 아니라 **부품 사이**에서 발생한다 — 보낸 쪽은 성공했다고 믿는데 받는 쪽에 두 번 도착한 메시지, 형식이 깨진 채 유입된 이벤트, 호출하려는 순간 중단된 상태인 이웃 서비스. 이 절은 그 세 가지를 일부러 일으킨다. 장애 대응 절차는 실제 장애 전에 시험되어야 한다는 것이 SRE의 오랜 원칙이다 (Beyer et al., 2016, Emergency Response) — 절차를 평시에 검증해 두어야 실제 장애에서 그대로 실행할 수 있다.

드릴의 계보: 카오스 엔지니어링의 공공 적용

이 절의 "고장을 일부러 낸다"는 발상은 이름이 있는 방법론이다 — **카오스 엔지니어링(chaos engineering)**. 먼저 정의를 확인한다. 카오스 엔지니어링은 "분산 시스템이 프로덕션의 격변을 견딘다는 확신을 세우기 위해, 시스템에 의도적으로 장애를 주입하고 실험하는 규율"이다(Principles of Chaos Engineering). 그 다섯 원칙은 이 장의 드릴 설계와 대부분 겹친다(③만 공공에서 굴절된다 — 아래): ① **정상 상태 가설**을 세우고(이 시스템에선 "마감은 확정되고 회계 보존식이 성립한다"), ② **실세계 사건**을 주입하고(중복 재전송·오염·의존 중단), ③ 가능하면 **프로덕션에서** 실험하고, ④ 실험을 **자동화**하고(오케스트레이터가 드릴을 매 실행 재현), ⑤ **폭발 반경(blast radius)**을 **최소화**한다(주입을 시나리오 3종·이벤트 몇 건으로 한정).

이 방법론의 계보는 민간에서 왔다. 넷플릭스는 프로덕션 인스턴스를 무작위로 종료시키는 Chaos Monkey와 그 확장군(Simian Army)을 상시 가동해, "언젠가 중단될 서버를 지금 중단시켜" 복원력을 강제로 검증했다(Netflix Technology Blog, *The Netflix Simian Army*). 왜 필요했나 — 없으면 무엇이 깨지는가. 복원력은 코드 리뷰나 시연으로 확인되지 않는다. 인스턴스가 실제로 중단되어야 "자동

복구가 실제로 작동하는지"가 드러나고, 그 확인을 사고 발생 시점이 아니라 계획된 시점에 수행하자는 것이 카오스 엔지니어링의 요지다. 넷플릭스 이후 이 실험을 상품으로 만든 도구들이 나왔다 — Gremlin, AWS Fault Injection Simulator, LitmusChaos 같은 것들이다. 도구가 맡는 일은 주입의 자동화와 폭발 반경의 제한이고, 공공에서 필요한 나머지 절반(계획된 시나리오와 감사 가능한 이력)은 도구가 아니라 운영 규칙이 만든다. 이 장 14.1이 "비기능 요구는 고장을 일부러 내 봐야 확인된다"고 한 명제가 바로 이 방법론의 요지다.

문제는 민간의 방식을 공공에 그대로 가져올 수 없다는 것이다. 넷플릭스도 임의의 시점에 무분별하게 중단시키는 것은 아니다 — 업무시간에, 모니터링과 대기 엔지니어를 갖춘 상태에서, 고객 영향을 최소화하는 조건으로 프로덕션 인스턴스를 무작위 종료한다(Netflix Technology Blog). 그럼에도 이 방식이 **프로덕션에서** 성립하는 것은 대규모 상용 서비스가 트래픽 여유와 즉시 롤백(rollback) 수단을 갖추고, 한 사용자의 짧은 부분적 열화를 감수할 수 있기 때문이다. 공공은 다르다. 민원·재난 시스템에서 무작위 주입은 특정 지역·특정 민원인을 무단으로 실험 대상으로 삼는 형평성 문제를 낳고(누구는 실험당하고 누구는 아닌가), 장애가 대민 서비스에 번지면 책임 소재가 남으며, 무작위성 자체가 사후 감사에서 "왜 하필 그때 그것을 켜는가"에 답할 수 없게 만든다. 그래서 공공 적용은 세 가지 변형을 거친다: **프로덕션 대신** 스테이징·드릴 환경에서, **무작위 대신** 계획된 시나리오로, **암묵적 실행 대신** 드릴 이력을 기록으로 남긴다. 이 장의 D1·D2·D3가 정확히 그 적용이다 — 폭발 반경을 셋으로 한정하고, 시점을 우리가 정하고(결정적 주입), 증거 JSON에 결과를 남긴다. 카오스 엔지니어링의 "프로덕션 무작위"가 공공에서 "계획된 드릴 + 감사 가능한 이력"으로 굴절되는 것, 그것이 공공 특수성이 민간 기법을 변형시키는 방식이다.

민간의 통합 운영 현장에서 흔히 요구되는 부분 실패 격리 패턴을 두 가지 적용 시나리오로 단순화하면(특정 기업의 공개 사례가 아니라 일반 관행을 재구성한 것이다) 이렇다. 이커머스의 주문 파이프라인은 결제 게이트웨이가 불안정해도 주문 접수를 중단하지 않는 방식으로 설계할 수 있다 — 결제 확정만 큐에 쌓아 두고 재시도하며, 게이트웨이가 돌아오면 수렴시킨다. 게임 라이브옵스에서는 한 지역 서버가 중단되어도 정산·랭킹 마감을 확정하도록 둘 수 있다 — 중단된 지역의 갱신만 대기로 남기고 나머지는 확정 보고한다. 두 사례의 공통 골격 — "확정 가능한 일은 확정하고, 특정 의존이 필요한 일만 잘라 대기로 남긴다" — 이 이 MVP의 공공 적용물이다: 모델 API가 중단되어도 집계는 확정하고 장애 중에도 보고는 계속되며(무중단 마감), 모델이 필요한 예측만 `pending_retry` 로 남긴다(드릴 D3). 민간의 이 부분 실패 격리 관행을 공공으로 옮길 때 더해지는 것이 하나 있다 — 장애 중에 확정된 보고가 "장애 중 산출물"임을 감사 기록에 남기는 일이다(민간은 매출만 맞으면 되지만, 공공은 "그 수치가 어떤 상태에서 만들어졌는가"가 검수 대상이다).

드릴 D1: 오염 이벤트 — 한 건이 전체를 멈추지 않는다

7/3의 정상 이벤트 22건 사이에 JSON으로 파싱되지 않는 페이로드 1건을 끼워 발행했다(경계가 아니라 한복판에 — 파이프라인이 "그 자리에서" 멈추지 않음을 봐야 한다). 실측: processor는 해당 메시지를 격리 큐(`quarantine.jsonl`)에 원문과 사유(`json_parse_error`)째 보존하고 다음 메시지로 넘어갔다. 격리 1건, 7/3 매핑 집계는 20건으로 온전했고 마감도 정상 확정되었다.

원칙은 2장에서 세운 그대로다 — 잘못된 데이터는 **폐기하지도, 통과시키지도 않는다. 분리해서 보존한다**. 격리 큐가 없으면 선택지는 둘뿐이다: 파이프라인이 중단되거나(가용성 상실), 오염이 집계에 섞이거나(정합성 상실). 격리는 셋째 길이며, 보존된 원문은 사후 분석(공격인가, 상류 버그인가)과 수리 후 재처리의 재료가 된다.

민간 메시징에서는 같은 장치를 DLQ(Dead Letter Queue, 처리에 실패한 메시지를 따로 모아 두는 큐)라고 부른다. 이름과 구현은 달라도 판단은 같다 — 실패한 메시지를 폐기하지 않고 분리한다. 운영에서 보는 지표도 같아진다. 격리 큐에 쌓인 건수와 그 건들이 머문 시간 두 가지다. 건수가 늘면 상류의 형식이 바뀐 것이고, 체류 시간이 길면 재처리 절차가 돌지 않는 것이다. 이 큐를 비우지 않으면 공공에서는 감사에서 누락된 민원이 되고, 민간에서는 처리되지 않은 주문이 된다.

드릴 D2: 중복 재전송 — 먹등이 수송 계층의 소음을 지운다

7/2의 이벤트 18건을 발행한 뒤, 마지막 10건을 한 번 더 발행했다 — 전송 확인(ack)을 받지 못한 생산자가 재전송하는 at-least-once의 재현이다(4장에서 실측한 그 동작, Kafka 공식 문서의 전달 보증 어휘). 실측: 물리 수신 28건, 고유 17건(중복 흡수 11건 — 재전송 10건 + 원천 파일 안의 중복 접수 1건), 매핑 집계는 16건으로 7장 확정치와 동일했다.

주목할 것은 흡수된 11건의 내역이다. 수송 계층의 재전송(10건)과 업무 계층의 중복 접수(1건)는 **원인이 다른데, 소비자 관점에서는 동일하게 나타난다** — 같은 complaint_id가 다시 도착한 것. 5장의 먹등 키(업무 식별자 기준 UPSERT)가 강력한 이유가 이것이다: 원인을 구분할 필요 없이 둘 다 무해화되고, seen_count에 관찰 흔적만 남는다. 회계 보존식(물리 28 = 고유 17 + 흡수 11, 고유 17 = 매핑 16 + 미기재 0 + 미매핑 1)이 “지워진 것이 아니라 흡수된 것”임을 증명한다.

같은 원칙이 소비자의 반대편 — 오프셋 커밋 — 에도 걸려 있다. processor는 오프셋 자동 커밋을 끄고, 메시지의 내구 쓰기(SQLite·파일)가 끝난 뒤에만 커밋한다. 자동 커밋은 처리가 끝나기 전에 오프셋을 밀어 올릴 수 있어, 크래시 시 그 메시지가 재생되지 않는다 — 4장의 어휘로 at-least-once가 at-most-once로 조용히 강등되는 것이다. “늦게 커밋하면 재생이 발생하지 않는가”가 다음 질문인데, 그 재생을 무해하게 만드는 것이 바로 위의 먹등 설계다: 재전송을 흡수하는 장치가 자기 자신의 크래시 재생도 흡수한다. 전달 보증과 먹등은 별개 기능이 아니라 한 쌍이다.

같은 설계를 민간에서는 주문 ID·거래 ID를 키로 삼아 쓴다. 여기서 중복이 흡수되지 않으면 결과가 곧바로 금액이다 — 같은 주문이 두 번 결제되거나 두 번 출고된다. 그래서 민간은 흡수 건수 자체를 지표로 본다. 전체 이벤트 대비 먹등 키로 흡수된 비율(중복 처리율)이 평소 수준을 벗어나면 상류의 재전송 로직이 바뀐 것이다. 이 MVP의 회계 보존식이 그 지표의 원자료다 — 7/2의 흡수 11건은 우리가 주입한 값이라 정상이지만, 주입하지 않은 날에 11건이 나오면 그것이 조사 신호다.

드릴 D3: 의존 서비스 중단 — 부분 실패의 격리와 복구

가장 중요한 드릴이다. 7/3 마감 직전에 모델 API 컨테이너를 정지시키고 마감을 진행했다. 실측:

```
[processor] 2026-07-03 마감: 물리수신 22 / 고유 22 (중복 흡수 0) → 매핑 20 · ... · 보존 True
[processor] 2026-07-03: 지연 레이블 도착 - 예측 3건 채점, 일 MAE 1.3333
[processor] 예측 2026-07-04 마포구: 실패(api_unreachable) → pending_retry
[processor] 예측 2026-07-04 관악구: 실패(api_unreachable) → pending_retry
[processor] 예측 2026-07-04 강남구: 실패(api_unreachable) → pending_retry
[processor] 2026-07-03: _SUCCESS 확정
```

마감은 중단되지 않았다. 집계 확정과 어제 예측의 채점(모델이 필요 없는 작업)은 완료되었고, 모델이 필요한 일(내일 예측)만 `pending_retry` 상태로 값이 되어 남았다. 이것이 **부분 실패 격리**다: "모델 API가 죽으면 마감 전체가 실패"가 아니라, 실패를 가장 작은 단위(예측 3건)로 잘라 상태로 기록하고 나머지는 확정한다. 9장에서 신호와 결정을 한 필드에 통합하지 않았던 것 (`metric_gate_pass`와 `promoted`의 분리), 7장에서 실패를 마커 부재로 남겼던 것과 같은 원칙의 세 번째 등장이다.

복구는 두 단계였다: 컨테이너 재기동(`/health` 정상 확인) 후, 재시도 제어 이벤트(`predict_retry`)를 발행하자 `processor`가 대기 3건을 재호출해 전부 성공했다(각 2회째 시도, 예측은 역시 6.0). 대기 0건 — 시스템이 장애 이전과 같은 상태로 **수렴**했다. 재시도가 안전한 이유도 설계에 있다: 예측 기록이 (대상일, 지역) 키의 멍등 쓰기라서, 재시도가 겹쳐도 중복 기록이 생기지 않는다. 복구 절차의 안전성은 복구 코드가 아니라 평시의 멍등 설계에서 온다.

민간에서 이 격리를 자동화하는 표준 부품이 서킷 브레이커(circuit breaker, 실패가 이어지는 의존 호출을 일정 시간 끊어 두는 장치)다. 이 MVP는 그 자리에 `pending_retry` 상태 기록을 두었다 — 호출을 끊지는 않되 실패를 값으로 남긴다. 호출량이 커지면 끊는 장치가 필요해지고, 그 항목은 표 14.5의 승격 목록과 같은 성격으로 들어간다. 계약의 언어도 함께 바뀐다. 민간은 이 설계를 가용률과 MTTR(Mean Time To Recovery, 평균 복구 시간)로 계약하며, 99.9% 가용률은 연간 8.7시간의 중단만 허용한다. D3의 재기동 → 재시도 → 대기 0건이 그 MTTR을 측정하는 구간이다.

표 14.6 장애 드릴 실측 요약

드릴	주입	기대(설계)	실측	판정
D1 오염 이벤트	파싱 불가 1건 (이벤트 한복판)	격리 후 계속	격리 1건(원문 보존), 7/3 매핑 20 온전, 마감 정상	통과
D2 중복 재전송	7/2 마지막 10건 재발행	집계 불변	물리 28→고유 17→ 매핑 16(7장과 동일), 보존식 성립	통과
D3 API 중단·복구	마감 직전 model- api 정지	집계 확정+예측만 대기→복구 수렴	<code>_SUCCESS</code> 확정, 대기 3건 (<code>api_unreachable</code>)→ 재시도 후 0건	통과

덧붙여, 이 장의 구현 과정에서 실제로 발생한 통합 오류 하나를 기록해 둔다. 처음 작성한 compose 파일은 processor와 ingester 두 서비스에 같은 이미지 태그의 빌드를 각각 선언했는데, docker compose build 가 둘을 병렬로 빌드하다 "image already exists" 충돌로 실패했다. 부품(각 Dockerfile)은 모두 정상인데 조립 선언이 경합을 유발한 것 — "통합의 고장은 연결부에서 난다"는 이 절의 명제가 실습 코드 자신에게 먼저 적용된 사례다. 수정은 빌드 선언을 한 서비스로 모으는 것이었다(compose 파일의 주석에 남겨 두었다).

운영 runbook 초안(정보 산출물)

runbook은 장애 순간에 판단을 대신해 주는 문서다 — 증상에서 출발해(원인이 아니라), 확인과 조치를 순서로 적는다. 사고 대응의 역할 분리와 기록 규율은 SRE의 사고 관리 프레임을 참고했다(Beyer et al., 2016, Managing Incidents). 아래 초안의 위 세 행은 이 장의 드릴로 **리허설이 끝난** 절차이고, 아래 행들은 실측 없이 절차만 문서화된 것이다 — 이 구분 자체를 runbook에 명시한다(검증된 절차와 미검증 절차는 신뢰 등급이 다르다).

표 14.7 운영 runbook 초안(민원 예측 MVP)

증상	1차 확인	조치	복구 확인	드릴
격리 큐에 신규 항목	quarantine.json의 사유·원문	상류 형식 오류면 발신처 통보, 공격 의심이면 보안 절차	격리 증가 중단·원천 수리 후 재처리	D1 완료
집계가 접수 체감보다 적음	품질 회계(중복 흡수·미매핑)	흡수는 정상(먹등), 미매핑 급증이면 코드 사전 갱신	보존식 성립 + 미매핑 감소	D2 완료
예측 누락·pending_retry	model-api 생존 (/health)·컨테이너 상태	재기동 → predict_retry 발행	대기 0건, 신원 정상 응답	D3 완료
마감 미완료 (_SUCCESS 부재)	processor 생존·Kafka 연결·day_close 발행 여부	processor 재기동(먹등 — 재마감 안전), 마감 재발행	해당 일 마커 생성	미실측
브로커 다운	컨테이너 상태·디스크	재기동(오프셋·상태는 보존), 유실 점검	소비 재개 + 회계 보존식	미실측
트래픽 급증(재난)	소비 지연(lag)·API 지연	12장의 부하 실측·스케일링 설계 선행 필요	—	12장
예측이 이상값	/health 신원(버전 확인)·입력 분포	배포 사고면 롤백(10장), 드리프트면 판단표(11장)	신원·분포 정상	10·11장

runbook을 실제로 읽는 장면: 야간 당직 워크스루

표를 채우는 것과 표로 일하는 것은 다르다. 드릴 D3의 실측을 재료로, 당직자가 표 14.7을 어떻게 소비하는지 재구성한다.

23시, 감시 리포트에 `pending_retry` 3건 이 기록되어 있다(증상). 당직자는 원인 추측 대신 표의 셋째 행을 찾는다 — 1차 확인: `/health` 호출. 응답이 없다(실측에서는 `api_unreachable` — 연결 자체가 거부됨). 이 시점에서 당직자가 **확인하지 않아도 되는 사항**이 두 가지다: 오늘 집계 손상되었는가? — 아니다, `_SUCCESS` 가 있으므로 확정본이다(부분 실패 격리). 예측이 유실되었는가? — 아니다, 대기 3건이 입력값째 상태로 남아 있다. 조치: 컨테이너 재기동, `/health` 정상 확인, 재시도 발행. 복구 확인: 대기 0건, 신원(champion v1) 정상. 소요 몇 분, 판단 없음 — 판단이 필요 없도록 만드는 것이 runbook의 목적이고, 그것이 가능했던 이유는 이 절차가 오늘 처음 실행된 것이 아니라 **드릴로 리허설된 것**이기 때문이다.

반대 경우는 다음과 같다. runbook이 없다면 당직자의 첫 행동은 대개 processor 재시작이다 — 증상(예측 누락)과 가장 가까워 보이는 부품이기 때문이다. 실제 원인(model-api)과 무관한 재시작은 상황을 바꾸지 못하고, 새벽의 두 번째·세 번째 추측이 이어진다. 증상→확인→조치의 표가 있는 조직과 없는 조직의 차이는 기술이 아니라 **새벽 2시의 인지 부하**다.

공공 사례

공공 시스템의 운영 인수인계에서 runbook의 유무보다 중요한 것이 **드릴 이력의 유무**다. “장애 대응 절차서 있음”은 문서 검사로 끝나지만, “최근 6개월 내 모의 장애 훈련 기록 있음”은 절차가 작동함의 증거다. 표 14.7의 마지막 열(드릴 완료/미실측)을 인수 문서에 포함시키는 것 — 그것이 이 장이 제안하는 검수 관행이다.

역방향: 이 장의 통합 설계를 민간 현장에 옮기면

지금까지는 민간 기법을 공공으로 옮겼다. 반대 방향도 성립한다 — 이 장에서 조립한 시스템을 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

이커머스 주문 파이프라인은 구조가 거의 그대로 겹친다. 주문 이벤트를 Kafka로 받고, 주문 ID 기준 먹등 소비로 중복 결제를 막고, 일별 마감으로 매출·환불·반품을 확정하고, 확정치로 익일 수요를 예측한다. 확정 → 채점 → 예측이라는 이 장의 주기가 매출 확정 → 예측 정확도 채점 → 익일 재고 예측으로 이름만 바뀐다. 도구도 이 책의 것과 겹친다 — Kafka-Airflow-MLflow에 Flink나 Spark Structured Streaming이 더해지는 정도다.

핀테크의 실시간 결제 사기 탐지는 스트림의 시계가 초 단위로 당겨진 경우다. 거래마다 점수를 매기고 임계를 넘으면 보류하므로, 일 마감이라는 접점이 사라지고 판정이 곧 집행이 된다. 그래서 D3가 여기서도 훨씬 무겁다. 모델이 죽었을 때 이 MVP처럼 `pending_retry` 로 미뤄 둘 수가 없고, 그 순간의 정책 — 전부 허용할 것인가, 전부 거부할 것인가, 최근 판정을 재사용할 것인가 — 을 미리 정해 두어야 한다. 공공의 “집행은 사람이 한다”는 경계(14.3)가 여기서도 성립하지 않고, 그 자리를 장애 시 기본 정책이 대신한다.

물류 추적은 거리 큐가 가장 바쁜 현장이다. 차량·선박의 위치 이벤트에는 GPS 오류와, 통신이 끊겼다 돌아올 때 몰려 들어오는 비정상 좌표가 섞인다. `_SUCCESS` 마커는 이 구간의 배송 상태가 확정되었다는 신호가 되고, 격리한 좌표를 뺀 나머지로 경로를 재구성한다. 거리 큐의 체류 지표(위 D1)를 실제로 매일 보는 곳이 이런 현장이다.

세 현장을 관통하는 것은 하나다. 계층 경계를 어디에 둘 것인가, 먹등 키를 무엇으로 잡을 것인가, 실패를 어느 단위로 자를 것인가, 확정을 무엇으로 신호할 것인가 — 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성·법적 의무라면, 민간에서는 매출·비용·SLA다. 인수 검수표도 같은 구조다. 민간에서는 같은 항목들이 CI/CD의 E2E 테스트 스위트가 되어 배포 전에 자동으로 돌고, 배포 성공률과 롤백 비율이 그 스위트의 커버리지를 되짚는 지표가 된다. 공공에서는 같은 표가 계약 이행의 증빙으로 남는다 — 사람의 승인 없이 자동 판정한다는 점은 같고, 판정 결과를 보관하느냐 흘려보내느냐가 다르다.

방향은 여기서도 비대칭이다. 민간 설계를 공공에 도입하면 증거 산출과 드릴 이력이라는 층이 하나 추가되지만, 공공 설계를 민간에 도입하면 그 층을 제외해도 손실이 없다 — 인수 검수표는 배포 게이트가 되고, 드릴 이력은 신뢰성 보고서가 되고, 회계 보존식은 정산 정확도가 된다.

핵심 정리

통합의 고장은 연결부에서 발생한다 — 오염은 격리하고(폐기하지도 통과시키지도 않는다), 중복은 먹등 키로 흡수하고, 의존 실패는 가장 작은 단위로 잘라 상태로 남긴다.

복구는 수렴으로 확인한다: 재기동 + 재시도 후 대기 0건 — 먹등 설계가 재시도를 안전하게 만든다.

runbook은 평시에 드릴로 검증한다 — 검증된 절차와 미검증 절차를 구분 표기하는 것까지가 문서화다.

격리·먹등·부분 실패 격리는 도메인을 가리지 않는 결정이다 — 공공은 감사·형평성이, 민간은 매출·비용·SLA가 같은 결정의 근거를 바꾼다.

실습 14.1 Docker Compose 기반 통합 MVP 실행(★★★★☆)

이 실습은 **설계** → **실행** → **대조** → **검수** 네 단계로 진행한다. 순서에 의도가 있다. 앞 13개 장이 정해 둔 값을 그대로 물려받아 compose를 기동하면 통합은 “실행되었다”로 끝난다. 통합에서 실제로 발생하는 것은 계층별 결정의 불일치인데(14.2), 그 불일치는 실행 로그에 불일치라고 기록되지 않는다. 먼저 결정들을 한 표에 모아 충돌을 찾고 드릴 결과를 예측한 다음, 실행 로그와 대조한다. 인수 검수는 마지막 단계에 그대로 둔다 — 설계 근거와 실행 증거가 모두 확보되어야 16항목을 해석할 수 있기 때문이다.

실습 목표

앞 장들이 정한 계층별 결정을 한 표로 모아 서로 충돌하는 지점을 찾는다(설계).
통합 MVP를 빌드·기동하고 3일치 운영 시나리오와 드릴 3종을 완주한다(실행).

드릴에서 어느 계층이 먼저 실패할지 예측하고 실측과 대조한다(대조).
 재조립 교차 검증 5종(표 14.3)과 인수 검수표 16항목(표 14.10)을 전향 통과시킨다(검수).

1단계 설계 — 계층 결정 일관성 점검과 드릴 예측

실행하기 전에 표 14.8의 빈칸을 직접 채운다. 값 자체는 앞 장들의 실습에서 이미 정했으므로 새로 만들 것이 없다. 이 표가 묻는 것은 그 값들을 한 시스템에 모았을 때 서로 버티는가다.

운영 조건(가상 발주 조건) - 구 단위 민원 접수를 실시간으로 받고, 하루가 마감되면 집계를 확정한다. - 확정 건수로 다음 날 예측을 만들고, 다음 날 실제 건수로 그 예측을 채점한다. - 재난 시 특정 구의 접수가 평시의 수십 배로 몰릴 수 있다. - 감사는 접수 누락·중복 흡수·모델 신원을 사후에 확인할 수 있어야 한다.

표 14.8 통합 설계 결정 일관성 점검표 — 학습자 작성용

계층	원 장이 정한 값	이 통합에서 쓸 값	왜 그 값인가	어느 결정과 충돌할 수 있는가
토픽·파티션 수 (4장)				
중복 제거 키(5장)				
창·마감 방식(6·7장)				
배치 주기(7장)				
승격 게이트(9장)				
감시 임계·판정 주기(11장)				
접근 등급(13장)				
이 조합이 깨지는 조건				

작성할 때 지킬 것이 셋이다. 첫째, 넷째 열(“왜 그 값인가”)에 “앞 장에서 그렇게 정했으니까”를 쓰지 않는다 — 그 값이 이 통합의 다른 결정과 어떻게 맞물리는지를 적는다. 둘째, 다섯째 열은 한 칸이라도 비워 두지 않는다. 충돌 후보가 없다고 판단했으면 왜 없는지를 적는다. 셋째, 마지막 행은 실행 뒤 3단계에서 다시 쓴다.

드릴 예측: 실행 스크립트는 드릴 3종을 주입한다. 실행하기 전에 각 드릴에서 어느 계층이 먼저 무너질 것 같은지 적어 둔다.

드릴	먼저 실패할 것 같은 계층	그렇게 본 근거	실제(3단계에서 기입)
D1 오염 이벤트 1건 주입			
D2 재전송 10건 주입			
D3 마감 직전 모델 API 정지			

“아무것도 실패하지 않는다”도 답으로 인정한다. 단, 무엇이 그것을 막는지를 근거 칸에 적어야 한다.

구성 요소와 실행 형태

compose 스택은 서비스 다섯 — zookeeper, kafka(3장 이미지 재사용), model-api(champion 고정 이미지), processor(상시 소비 서비스), ingester(단발 발행 도구) — 로 이루어진다.

Kafka-Zookeeper의 호스트 포트는 공개하지 않으므로(내부 네트워크 전용) 3장 로컬 스택이 떠 있어도 충돌 없이 공존한다. 호스트에 노출되는 것은 모델 API의 8014 포트 하나다.

compose 파일은 짧지만, 줄마다 통합의 설계 결정이 들어 있다. 다섯 개만 분석하면 —

기동 순서는 시간이 아니라 상태로: processor는 kafka의 `service_healthy` (브로커 API 응답 확인)를 기다린다. “10초 기다리면 된다”는 식의 시간 의존은 기계가 느린 날 실패한다 — 7장의 마커, 10장의 `/health`와 같은 원칙(상태를 물어라)의 compose판이다.

토픽 자동 생성 금지: `KAFKA_AUTO_CREATE_TOPICS_ENABLE=false`. 이 시스템의 마감 순서 보장은 “단일 파티션” 전제 위에 서 있는데, 자동 생성은 그 전제를 조용히 깰 수 있다. 전제는 설정으로 잠그고, 토픽은 파티션 수를 명시해 만들며, 이미 있으면 파티션 수를 검증한다(침묵 무동작 금지 — 4장 gotcha).

최소 노출: 브로커에 호스트가 직접 붙을 일이 없으므로 리스너를 내부 하나로 줄였다. 노출면 축소는 보안이기 이전에 충돌 회피다(3장 스택과의 공존이 그 증거).

의존이 정지를 전파하지 않는다: 드릴 D3에서 model-api를 정지시켜도 processor는 계속 동작한다 — compose의 `depends_on`은 기동 순서일 뿐 생존 결합이 아니고, 생존 결합을 끊는 것은 processor 쪽의 실패 격리 설계다.

빌드 선언은 한 곳에: 같은 이미지 태그를 두 서비스가 각자 빌드 선언하면 병렬 빌드 경합으로 실패한다(14.5에서 기록한 이 장의 실제 오류). 이미지 하나 = 빌드 선언 하나, 나머지 서비스는 참조만.

정본 목차가 예고했듯 이 실습의 표준 경로는 **사전 빌드된 이미지에서 실행**하는 것이다: 이미지 빌드는 1회의 사전 단계이고(첫 빌드는 네트워크 상태에 따라 수 분), 이후의 실행·재실행은 캐시된 이미지로 빠르다(이번 실측에서 재빌드 판정 2초). 강의 환경에서는 조교가 빌드를 마친 이미지를 `docker save/load (tar)`로 배포할 수 있다 — 10장에서 세운 폐쇄망 배포 경로와 같은 원리이며, 실측 이미지 크기는 model-api 278MB(10장 단독 이미지와 동일 크기), pipeline 48MB다(둘 다 환경·시점에 따라 변동). 공개 레지스트리를 통한 배포는 저장소 인프라 확보 시의 선택지로 남긴다.

2단계 실행 — 선수 조건과 실행

```
cd practice/chapter13
python3 -m venv venv && source venv/bin/activate # (Windows: venv\Scripts\activate)
pip install -r code/requirements.txt
python run_chapter13.py # Docker Desktop 실행 상태여야 한다(부록 A)
```

오케스트레이터가 전 과정을 자동 수행한다. 단계별로 무엇을 보게 되는지 정리하면 —

단계	화면에서 보게 되는 것	여기서 확인되는 요구
0 사전 점검·bootstrap	docker 버전, 훈련 쌍 6개·지문 96f6b6b3..., champion v1 승격·export 검증 6.0, sha256 일치	재조립 증명(표 14.3의 1·2)
1 초기화·빌드	이전 스택·상태 완전 제거(먹등 재실행), 이미지 2종 크기	폐쇄망 배포물(표 14.1)
2 기동·스모크	/health 신원 응답, 6.0 예측, 422 거절	서빙 계약(표 14.10의 07~09)
3 3일 시나리오	일별 마감 로그(물리/고유/매핑), 예측·채점 로그, 드릴 D1·D2의 흔적	기능 요구 전부 + 먹등·격리
4 드릴 D3	중단 중 마감 확정, 대기 3건, 복구 수렴 0건	부분 실패 격리·복구
5 리포트·검수표	16항목 [PASS] 목록, CH14_RUN_PASS	인수(표 14.10 전체)

재실행은 안전하다 — 단계 1이 이전 실행의 컨테이너·볼륨·런타임 산출물을 매번 삭제하고 시작하므로, 몇 번을 실행해도 증거 JSON은 바이트까지 같다(이번 집필에서 2회 실행으로 확인). 이 장의 실측 환경: Python 3.13 venv(mlflow 3.14.0, scikit-learn 1.9.0 — 10장과 동일), Docker 28.0.4, 컨테이너는 python:3.13-slim 기반에 kafka-python 3.0.7(4장 실측 버전) 고정.

흔한 설치·실행 문제와 대응(모두 오케스트레이터가 명시적 메시지로 안내하도록 설계되어 있다):

증상	원인	대응
<code>docker</code> 데몬에 연결할 수 없다 후 즉시 종료	Docker Desktop 미실행	Docker 실행 후 재시도(부록 A)
첫 빌드가 수 분 걸림	베이스 이미지 의존성 다운로드	정상 — 사전 빌드 이미지(tar) 배포가 표준 경로인 이유
<code>8014</code> 포트 충돌	다른 프로세스가 사용 중	compose의 ports 매핑 변경(왼쪽 숫자만)
<code>_SUCCESS</code> 생성 대기 시간 초과	processor 이상	오케스트레이터가 processor 로그의 마지막 부분을 자동 출력 — 그 부분부터 읽는다
지문 불일치로 bootstrap 실패	입력 parquet 훼손·교체	9장 스냅샷과 다른 데이터라는 뜻 — 입력을 저장소 원본으로 복원

핵심 코드

부분 실패 격리 — 마감(확정 가능한 작업)과 예측(재시도 가능한 작업)의 분리:

```
resp, err, ms = call_predict(r["lawd_cd"], r["count"])
if resp is None:                                # 모델 API 실패 - 마감은 계속 간다
    row = (target, r["lawd_cd"], ..., "pending_retry", 1, err) # 실패를 값으로 기록
...
(out / "_SUCCESS").write_text("")                # 마커는 마지막에(7장) - 집계는 확정된다
```

전체 코드는 `practice/chapter13/code/13-1-integrated-mvp/` 참고
(`processor.py`, `ingester.py`, `monitor.py`, `docker-compose.yml`)

실행 중 확인 체크리스트

- ☐ bootstrap이 지문 `96f6b6b3...` 을 출력하고 `CH14_BOOTSTRAP_OK` 로 끝나는가(9장 스냅샷 일치)
- ☐ 서빙 앱 재조립 확인: `sha256` 일치 가 출력되는가(10장 `app.py`와 바이트 동일)
- ☐ 스모크에서 `predict`(전일 9) → 6.0 과 잘못된 입력 → 422 가 나오는가(10장 실측 재현)
- ☐ 2일차 마감 로그가 물리수신 28 / 고유 17 (중복 흡수 11) → 매핑 16 인가(드릴 D2)
- ☐ 드릴 D3에서 마감 확정(`_SUCCESS`) + 예측 대기 3건 후 복구 수령: 대기 0건 인가
- ☐ 인수 검수표 16항목이 모두 `[PASS]` 이고 마지막 줄이 `CH14_RUN_PASS` 인가
- ☐ 실행 전에 표 14.8과 드릴 예측표를 적어 두었는가(실행 후에 적으면 대조가 성립하지 않는다)

실행 결과(실제)

콘솔 출력 발췌(전체는 `data/output/ch13_run_log.txt` — 이 실행의 보존본은 `ch13_run_log_preserved.txt`):

```

health={'status': 'ok', 'model_name': 'complaint_daily_forecaster',
        'model_version': '1', 'model_source': 'local-dir', 'shadow_enabled': False}
predict(전일 9) → 6.0 (4.7ms) / 잘못된 입력 → 422
...
[processor] 2026-07-02 마감: 물리수신 28 / 고유 17 (중복 흡수 11) → 매핑 16 · 미기재 0 · 미매핑 1 ·
보존 True
[processor] 2026-07-02: 지연 레이블 도착 - 예측 3건 채점, 일 MAE 0.6667
...
[processor] 재시도 성공 2026-07-04 강남구: 입력 9 → 6.0 (1.9ms, 2회째)

```

표 14.9 3일 운영 시뮬레이션 실측(일별 회계·집계·예측·채점)

항목	7/1	7/2 (드릴 D2)	7/3 (드릴 D1-D3)
발행(정상+주입)	20	18+재전송 10	22+오염 1
물리 수신(민원 이벤트)	20	28	22
고유(먹등 흡수 후)	20	17	22
중복 흡수	0	11(재전송 10+중복 접수 1)	0
매핑/미기재/미매핑	18/1/1	16/0/1	20/1/1
회계 보존식	성립	성립	성립
확정 집계(마포·관악·강남)	5·5·8	5·5·6	6·5·9
7장 확정치와 일치	예	예	예
익일 예측(3개 구)	전부 6.0	전부 6.0	중단→대기 3건→복구 후 6.0
전일 예측의 채점(일 MAE)	—	0.6667	1.3333
격리(오염)	0	0	1

지연 채점의 손 계산 — 7/2: 예측은 세 구 모두 6.0, 실제로는 5·5·6이므로 $(|5-6|+|5-6|+|6-6|)/3 = 2/3 \approx 0.6667$. 7/3: 실제 6·5·9이므로 $(0+1+3)/3 = 4/3 \approx 1.3333$. 채점된 6건 전체로는 $(1+1+0+0+1+3)/6 = 1.0$ — 9장 baseline의 훈련 MAE와 정확히 같다(같은 데이터·같은 모델이므로 필연이고, 감시 경로의 정합성 검증이다). 7/4를 겨냥한 예측 3건은 실제 건수가 아직 없어 미채점으로 남았다 — 11장의 “레이블 지연”이 이 시스템에서는 “다음 날 마감 전까지”라는 구체적인 길이를 갖는다.

관찰 하나를 덧붙이면, 예측 호출 지연의 첫 호출은 11.5ms였고 이후 호출은 1.4~4.9ms였다(보존 로그 기준 — 값은 실행마다 변동). 4장의 첫 레코드에서 110.7ms로 확인한 콜드스타트가 API에서도 나타났다. 이는 평균뿐 아니라 상위 분위수를 확인해야 한다는 11장의 원칙이 다시 나타난 사례다.

3단계 대조 — 예측과 실측의 대조

1단계에서 적어 둔 드릴 예측표의 “실제” 칸을 표 14.9과 표 14.6의 값으로 채운다. 예측이 어긋나는 자리가 셋인데, 어긋나는 방식마다 확인할 내용이 다르다.

D1을 집계 오차로 예측했다면 격리 큐를 고려하지 않은 것이다. 실측에서 7/3 매핑은 20건으로 온전했고, 오염 1건은 집계 밖의 파일에 남았다. 오염이 집계를 오염시키는 것은 격리 큐가 없을 때뿐이다 — 있으면 오염은 계층을 손상시키지 않고 별도 경로로 분리된다.

D2를 집계 증가로 예측했다면 물리 수신과 고유를 같은 것으로 본 것이다. 실측은 물리 28 → 고유 17 → 매핑 16이고, 7장 확정치와 같다. 늘어난 것은 수신 건수뿐이고 집계는 변하지 않았다. 여기서 배울 것은 중복이 없었다는 사실이 아니라, 중복이 있었는데도 값이 같다는 것을 회계로 증명할 수 있다는 점이다.

D3를 마감 실패로 예측했다면 그 예측이 오히려 대부분의 시스템에서 맞다. 이 MVP에서 마감이 살아남은 것은 윤이 아니라 확정 가능한 일과 모델이 필요한 일을 나눠 둔 설계 때문이다. 예측이 어긋난 지점이 곧 설계가 수행한 역할이다 — 나누지 않았다면 예측대로 마감 전체가 실패했을 것이다.

그다음 표 14.8으로 돌아가 마지막 행(“이 조합이 깨지는 조건”)을 다시 쓴다. 실행 전에는 추측이었지만 지금은 근거가 있다. 파티션을 3개로 올렸을 때 일 마감이 어느 조건에서 틀리는지, 표본 3건짜리 MAE로 게이트를 판정하면 7/3의 1.3333이 무엇을 유발하는지를 실측 옆에 적을 수 있다. **제출하는 설계표는 이 수정을 거친 최종본이다.**

4단계 검수 — 인수 검수표(정본 산출물)

monitor가 실행 증거에서 자동 생성한 검수표다(`ch13_acceptance_checklist.json`). 각 항목은 주장이 아니라 계산된 불리언이다 — ML 시스템의 프로덕션 준비도를 “검증 가능한 항목의 목록”으로 명세하는 접근(Breck et al., 2017)을 이 시스템의 크기로 축소한 것이며, 10장의 배포 전 체크리스트(8항목)가 통합 수준으로 확장된 형태다.

ML Test Score의 원 논문은 준비도 테스트를 **네 범주**로 나눈다 — 데이터·피쳐 테스트, 모델 개발 테스트, ML 인프라 테스트, 모니터링 테스트 — 그리고 각 테스트를 수동 수행 0.5점·반복 자동화 1점으로 계산해 범주별로 합산한 뒤, 그 **네 값의 최솟값**을 최종 점수로 삼는다(Breck et al., 2017 — 가장 약한 범주가 준비도를 지배한다는 뜻이다). 이 16항목 검수표를 그 렌즈로 재정렬하면 대응이 드러난다: 02·03·04·15가 **데이터·피쳐 테스트**(집계·정제 회계·보존식·피쳐 지문), 07·13이 **모델 테스트**(예측 재현·평가 정합), 01·05·06·08·09·10·11·14·16이 **인프라 테스트**(E2E·맥동·격리·신원·입력 검증·부분 실패·복구·서빙 동일성, 그리고 드릴 리허설은 원 논문의 롤백·비상 절차 시험에 대응), 12가 **모니터링 테스트**(지연 평가 기록)에 든다. 이 대응이 곧 이 검수표의 한계도 보여 준다 — 특히 **모니터링 범주가 지연 평가 기록 한 항목으로 가장 얇다**. 원 논문이 최솟값을 최종 점수로 삼는 논리를 적용하면, 이 시스템의 준비도는 바로 이 얇은 모니터링 범주가 결정한다. ML Test Score의 만점 루브릭이 요구하는 “의존성 변경 알림·훈련·서빙 스큐·성능 회귀의 상시 자동 감시”는 이

MVP의 크기 밖이며(단일 시나리오·소표본), 그 확장이 표 14.5의 제외 항목들(감시 시각화·알림)이 돌아오는 지점이다. 적용 자격을 밝히면 — 이는 ML Test Score의 루브릭을 방법론으로 차용해 검수표를 조직한 것이지, 원 논문의 점수 산정 절차를 그대로 집행한 것은 아니다.

표 14.10 통합 시스템 인수 검수표(전 16항목 실측 통과)

#	검수 항목	검증 근거(자동 대조)	결과
01	E2E 경로 3일 마감	_SUCCESS 마커 3개	통과
02	집계가 7장 확정치와 일치	지역별 건수 3일 대조	통과
03	정제 회계가 7장과 일치	매핑·미기재·미매핑·보정 건수 대조	통과
04	회계 보존식 성립	물리=고유+중복, 고유=매핑+미기재+미매핑	통과
05	중복 재전송 무해	물리 28 → 매핑 16 불변	통과
06	오염 이벤트 격리	격리 1건 + 파이프라인 지속	통과
07	예측이 9·10장 실측과 일치	호출 9건 모두 6.0	통과
08	서빙 신원 기록	전 응답 champion v1·local-dir	통과
09	잘못된 입력 거절	422 응답	통과
10	부분 실패 격리	중단 중 마감 확정 + 예측 대기 3건	통과
11	복구 수렴	재시도 후 대기 0건·ok 9건	통과
12	지연 평가 기록	채점 6건·일별 MAE 산출	통과
13	지연 MAE = 9장 훈련 MAE	전체 1.0 일치(손 계산)	통과
14	서빙 앱 바이트 동일	10장 app.py sha256 일치	통과
15	피쳐 지문 일치	9장 지문 96f6b6b3... 일치	통과
16	장애 드릴 3종 리허설 완료	D1·D2·D3 증거 존재	통과

검수표를 읽는 법: 01~04가 기능 요구의 인수(표 14.1의 앞 절반), 05~11이 비기능 요구의 인수(뒷 절반), 12~13이 감시·평가 연결, 14~16이 재조립 증명과 운영 준비다. 발주 문서의 요구가 검수표의 항목으로, 항목이 자동 대조 코드로 이어지는 이 사슬이 — “인수 가능한 시스템”의 실체다.

산출물 안내

`data/output/ch13_integration_report.json`: 통합 리포트(일별 회계·예측·지연 평가·드릴 실측 — 재실행 바이트 동일 확인)

`data/output/ch13_acceptance_checklist.json`: 인수 검수표(정본 산출물 — 자동 생성)

`data/output/ch13_bootstrap_summary.json` · `ch13_smoke.json`: 재구성·스모크 증거

`data/output/ch13_run_log_preserved.txt`: 이 장 집필 시점 실행의 콘솔 보존 기록(지연 등 변동 값의 원천 — 재실행 산출물 아님)

`data/output/mvp/`: 런타임 산출물(일별 확정 집계·예측 스냅샷·격리 큐·상태 DB — 커밋 제외)

표 14.8 계층 결정 일관성 점검표와 드릴 예측·실측 대조표(학습자 작성 — 3단계 대조 후 확정)

실습과 운영의 간극(축소 지점 목록)

3일·3개 구·60건의 시뮬레이션 민원이다 — 운영은 연속 운전이고, 규모가 커지면 표 14.5의 제외 항목들이 차례로 필요해진다.

일 마감을 오케스트레이터가 발행한다 — 운영은 스케줄러(7장)가 발행하고, 마감 지연·누락 감시가 붙는다.

재시도를 제어 이벤트로 수동 트리거했다 — 운영은 백오프를 둔 자동 재시도와 한도 초과 시 사람 호출이 표준이다.

장애 주입이 결정적(우리가 정한 시점)이다 — 실제 장애는 예고가 없으며, 그래서 정기 드릴이 필요하다(14.5).

인수 검수표가 “이 시나리오”의 통과를 보증할 뿐이다 — 시나리오 밖 고장(브로커 다운·디스크 고갈)은 runbook의 미실측 행으로 남아 있다.

위험 식별(평가표 연계)

위험	이 실습에서의 증거	운영 대응
조용한 중복 오염	물리 28과 매핑 16 — 회계로만 확인됨	회계 보존식을 감시 지표로 상설화
오염 이벤트로 파이프라인 정지	격리 큐 부재 시 D1은 정지 또는 오염	격리 큐 + 격리율 알림
의존 실패의 전면화	D3에서 격리 성공(설계에 의한 결과)	확정/재시도 분리를 설계 표준으로
재조립 시 부품 변형	sha256·지문·확정치 자동 대조	교차 검증을 CI 게이트로(10장)
계층 결정의 불일치	표 14.8 점검: 파티션 3개 ↔ 일 마감 순서 전제, 게이트 1.05 ↔ 하루 표본 3건	결정표를 통합 산출물로 유지하고, 어느 계층의 값을 바꿀 때 충돌을 재점검
검수표의 형식화	전 항목이 증거 계산식	“주장 항목”을 검수표에서 배제

제출물 3종: ① 표 14.8 계층 결정 일관성 점검표(3단계 수정을 반영한 최종본), ② `ch13_acceptance_checklist.json`, ③ 드릴 예측·실측 대조표. ①이 설계, ②가 증거, ③이 둘을 이은 대조 기록이다.

보충(전문가 트랙 포인터)

재난 상황의 트래픽 급증 대응: 이 MVP의 한계선이다. 평소 60건이 재난 시 수만 건이 될 때 어디가 먼저 포화하는지(소비 지연인가, API 포화인가)는 추측이 아니라 부하 실측의 문제다 — 12장의 Locust 방법론으로 한계를 재고, 스케일링(파티션 확장·서빙 복제)을 설계한 뒤에야 “재난 시스템”을 말할 수 있다.

운영 인수인계 문서와 runbook 심화: 사고 관리의 역할 분리(지휘·소통·실행)와 사후 포스트모템(postmortem) 규율은 SRE 문헌의 사고 관리 장(Beyer et al., 2016)이 출발점이다. 17장의 포스트모템 실습이 이 연장선에 있다.

공공 조달·운영 계약 관점의 산출물: 이 장의 산출물 4종(코드·runbook·검수표·검토표)을 조달 문서의 어휘(과업지시서의 산출물 목록, 검사 기준)로 전환하는 작업 — 부록 D가 장별 검수 자산의 색인이다.

정책 방법론 Lab과 평가지표 주입: 게이트 값(1.05)·감시 임계·재학습 조건을 누가 정하고 개정하는가. 운영 조직 밖의 평가·검증 조직이 기준을 관리하는 구조는 17장에서 다룬다.

다중 파티션에서의 일 마감: 이 MVP는 단일 파티션으로 마감 순서를 보장했다. 파티션이 여럿이면 마감 신호가 모든 파티션의 해당 일 이벤트 “뒤”에 온다는 보장이 사라진다 — 6장의 워터마크가 정확히 이 문제의 스트리밍 해법이며, 파티션별 마감 정족수 같은 설계가 필요해진다.

핵심 용어

통합 MVP: 핵심 경로만 포함하고 제외 범위를 명시한 최소 운영 시스템 — 가치는 작음이 아니라 경계의 명시

연결부 고장: 부품 사이에서만 생기는 고장(중복 재전송·오염 이벤트·의존 서비스 중단)

격리 큐(quarantine): 오염 이벤트를 폐기하지도 통과시키지도 않고 분리 보존하는 장치

부분 실패 격리: 실패를 가장 작은 단위로 잘라 상태로 기록하고 나머지는 확정하는 설계

일 마감(day_close): 스트림 수신과 배치 확정을 잇는 제어 이벤트 — 두 시계의 접점

지연 채점: 다음 날 확정 건수(레이블)로 전날 예측을 자동 채점 — 11장 지연 평가의 실체화

재조립 교차 검증: 부품의 커밋 실측과 통합 후 실측의 자동 대조(지문·해시·확정치·예측·MAE)

인수 검수표: 실행 증거에서 자동 생성되는 불리언 항목 목록 — 주장이 아니라 계산

드릴: 장애 대응 절차의 평시 리허설 — runbook의 검증 절차

검수 질문 한 줄 요약

발주·검수 문서에서 이 장은 세 질문으로 압축된다: ① 비기능 요구(유실·중복·부분 실패)마다 **검증 방법**이 명세되어 있고 그 검증이 실제 수행되었는가(표 14.1→14.9의 사슬 — 시연 통과는 기능의 증거일 뿐이다), ② 장애 대응 절차가 문서로 있을 뿐 아니라 **드릴로 리허설**되었는가(표 14.7의 드릴 열 — 검증된 절차와 미검증 절차의 구분 표기), ③ 인수 검수표의 항목들이 사람의 주장인가 실행 증거의 계산인가(자동 대조 코드의 존재). 셋 다 없으면 그 통합 시스템의 인수는 시연회이지 검수가 아니다.

다음 장 예고

이 장으로 예측 모델의 운영 루프 — 데이터에서 배포, 감시, 재학습까지 — 가 하나의 시스템으로 닫혔다. 15장은 이 루프에 새 종(種)의 모델이 들어올 때를 다룬다: LLM이다. 민원 요약·답변 초안·정책 질의응답 같은 공공 LLM 서비스는 이 장의 어휘(버전·게이트·감시·격리)를 그대로 요구하면서, 새 문제를 더한다 — 출력이 확률적 문장이라 “예측 6.0 = 기대값 6.0” 같은 대조가 불가능하고, 근거(출처)의 추적과 환각의 통제가 검수 항목이 된다. 기존 MLOps가 LLMOps로 확장되는 지점과 달라지는 지점을 구분한다.

참고문헌

Docker. (n.d.). *Docker Compose* — *Docker Docs*. <https://docs.docker.com/compose/>
Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering* — Ch. 13: Emergency Response, Ch. 14: Managing Incidents. O'Reilly. <https://sre.google/sre-book/emergency-response/> · <https://sre.google/sre-book/managing-incidents/>
Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. *IEEE International Conference on Big Data 2017*. <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production->

readiness-and-technical-debt-reduction/

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *NeurIPS 2015*.

https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fc2674f757a2463eba-Abstract.html

Apache Kafka. (n.d.). *Documentation — Message Delivery Semantics*.

<https://kafka.apache.org/documentation/>

행정안전부. (시행 중 행정규칙). 행정기관 및 공공기관 정보시스템 구축·운영 지침.

국가법령정보센터. <https://www.law.go.kr/행정규칙/>

행정기관및공공기관정보시스템구축 · 운영지침

Principles of Chaos Engineering. (n.d.). *Principles of Chaos Engineering*.

<https://principlesofchaos.org/>

Netflix Technology Blog. (2011). *The Netflix Simian Army*. Medium.

<https://netflixtechblog.com/the-netflix-simian-army-16e57fbab116>

Fowler, M. (2011). *ContractTest*. martinowler.com.

<https://martinowler.com/bliki/ContractTest.html> (소비자 주도 계약 구현: Pact Docs,

<https://docs.pact.io/>)